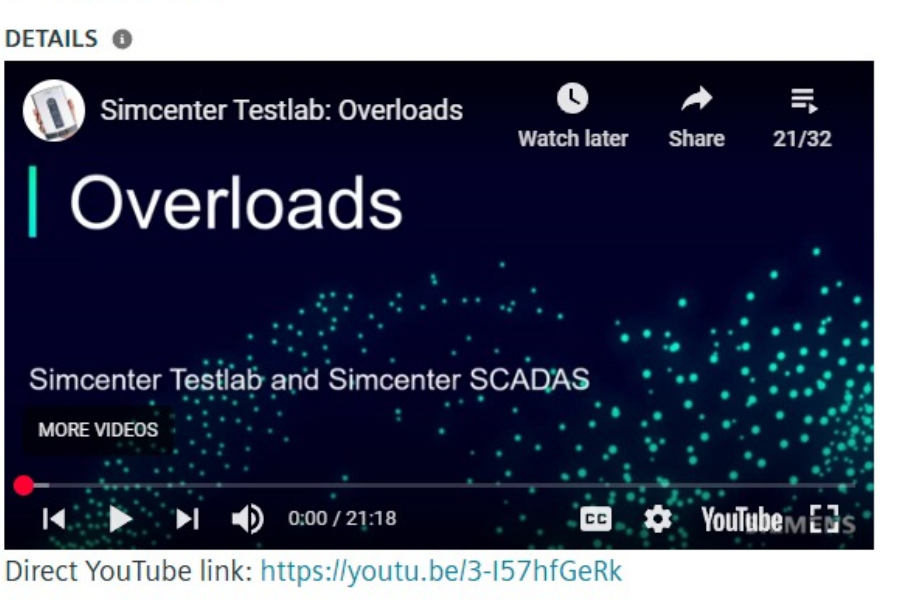
Introduction to Filters: FIR versus IIR

6-Apr-2020 Knowledge

DETAILS


Simcenter Testlab: Overloads
Watch later
Share
21/32
Overloads
Simcenter Testlab and Simcenter SCADAS
MORE VIDEOS
0:00 / 21:18
YouTube

Direct YouTube link: https://youtu.be/3-l57hfGeRk

****** Free Seminars: Digital Data Acquisition and Signal Processing ******

Filters have a variety of applications in data acquisition and analysis. They are used to alter the frequency content of a time signal by either reducing or amplifying certain frequencies.

For example, as shown in *Figure 1*, a low pass filter affects frequency content in a signal in three different ways: Some frequency content remains unchanged, while other frequency content is either reduced in amplitude or removed entirely from the signal.

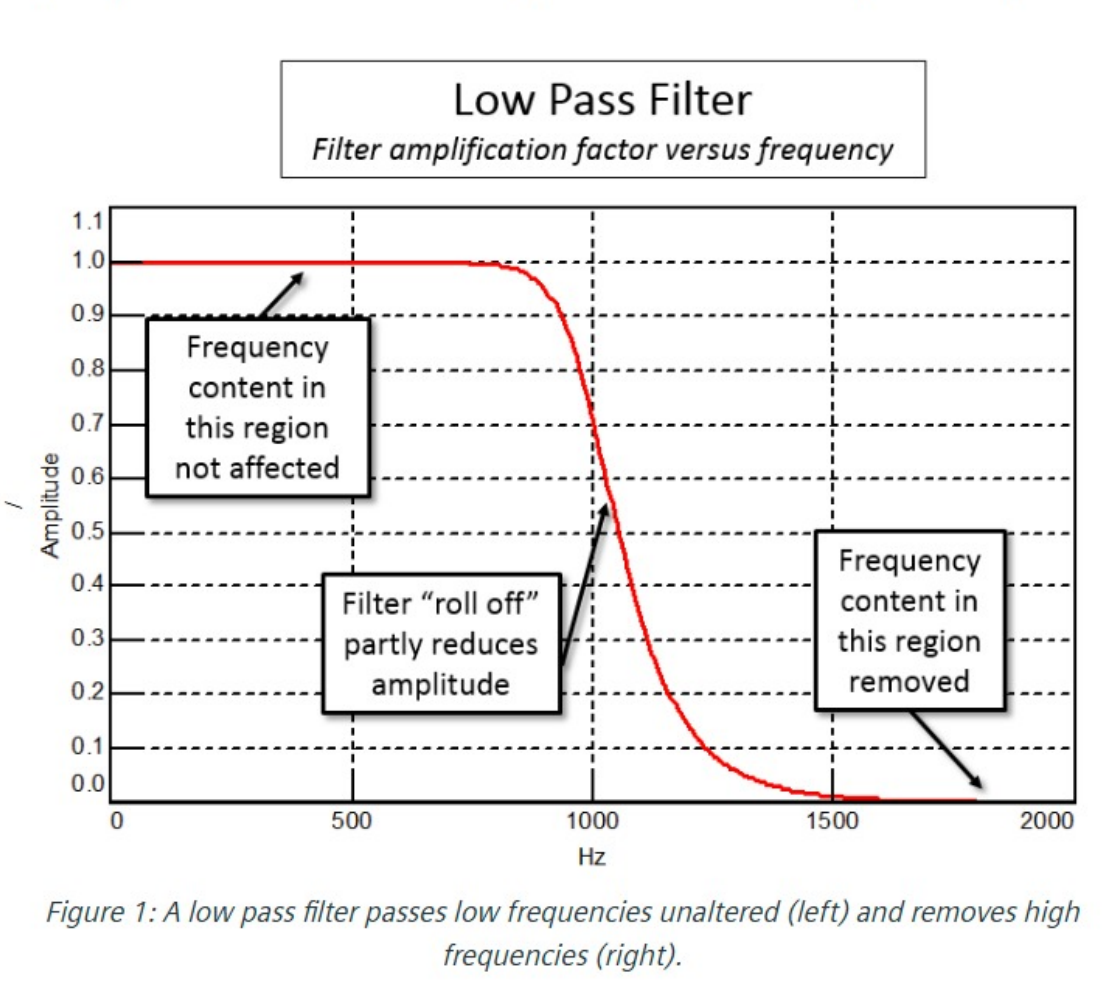


Figure 1: A low pass filter passes low frequencies unaltered (left) and removes high frequencies (right).

Filters can also amplify frequency content, not just reduce or remove it. The amount that a filter adjusts the amplitude of a signal can be expressed in either linear terms (i.e., amplification factor) or decibels of gain/attenuation as shown in *Figure 2*.

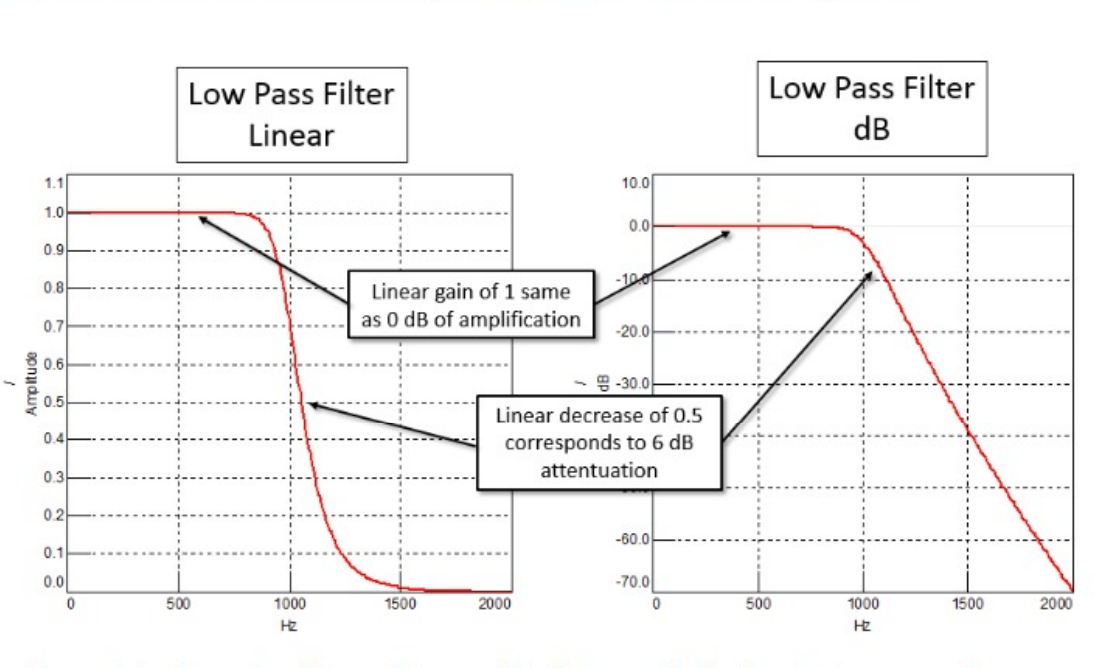


Figure 2: Left graph – Linear filter amplitude in multiplication factor versus frequency, Right graph – Same filter in decibel scale of attenuation versus frequency.

The linear scale has the following equivalents in decibel scale:

- Halving of linear amplitude is 6 dB of attenuation
- No adjustment of amplitude corresponds to linear gain of 1, or 0 dB, of attenuation

While useful to view filter characteristics in the frequency domain, filters perform their work in the time domain (*Figure 3*).

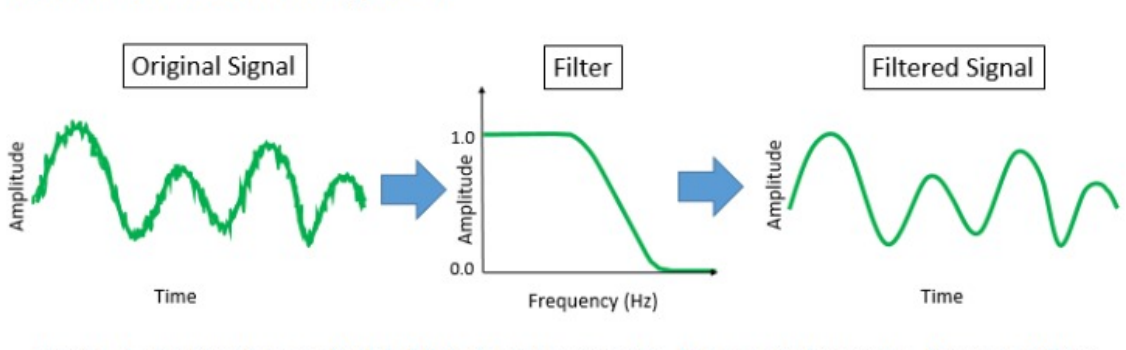


Figure 3: An input signal with high frequency noise is passed through a low pass filter. The resulting output has the high frequency noise removed, resulting in a clean signal.

A filter takes a time domain signal as input, modifies the frequency content, and outputs a new time domain signal. This can be useful in a variety of applications.

Contents of this article:

1. Applications of Filters
2. Filter Types
3. FIR versus IIR
4. Filter Order and Computational Speed
5. Time Delay and Filters
  5.1 Direct versus Zero Phase
6. Filter Methods and Attributes
  6.1 FIR Filter Methods
  6.2 IIR Methods
7. Filtering in Simcenter Testlab

1. Applications of Filters

Filters are used in different ways, from signal cleanup to analysis.

In some applications, filters are used to condition a time domain signal by attenuating unwanted frequency content. Examples include:

- Anti-Aliasing Filter – An anti-aliasing filter is used to remove signal content that cannot be properly digitized before Analog-to-Digital conversion.
- Noise Removal – Filters could be used to remove unwanted high frequency noise from a signal, for example, a hiss in a musical recording.
- Drift Removal – Drift or large offsets can be removed from a signal via a high pass filter or AC coupling.

Sometimes filters are used to introduce specific characteristics into signal data for analysis:

- A-Weighting – An A-weighting filter (*Figure 4*) is used on microphone data to introduce characteristics of human hearing into the microphone recording. A microphone "hears" all frequencies equally well, but the human ear does not. The A-weighting filter reduces the high and low frequencies of a microphone signal to reflect how the human ear perceives sound.

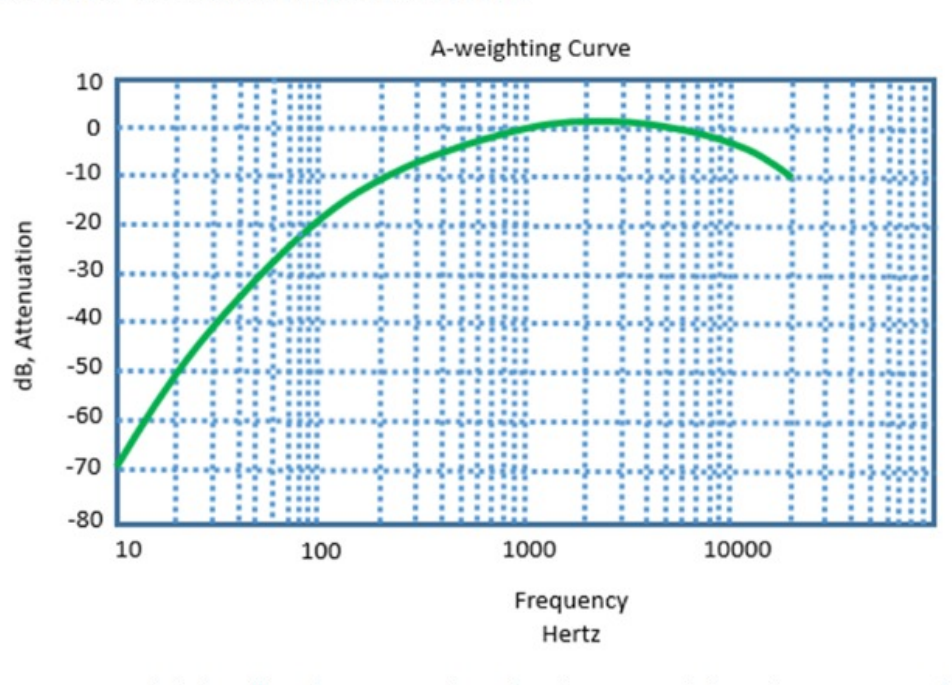


Figure 4: A-weighting filter frequency domain shape. Y-axis is gain or attenuation in dB versus X-axis in frequency.

- Human Body Vibration – The human body is more sensitive to certain vibration frequencies than others. A filter (defined by ISO 2631) can be used to evaluate human body health and comfort based on an accelerometer vibration signal.

Filters can be implemented in analog (for live signals) systems or digitally (for manipulation of an already recorded signal on a PC).

2. Filter Types

Filters can be designed for different tasks. For example, a filter could be classified as either a high pass, low pass, band stop, or band pass (see *Figure 5*):

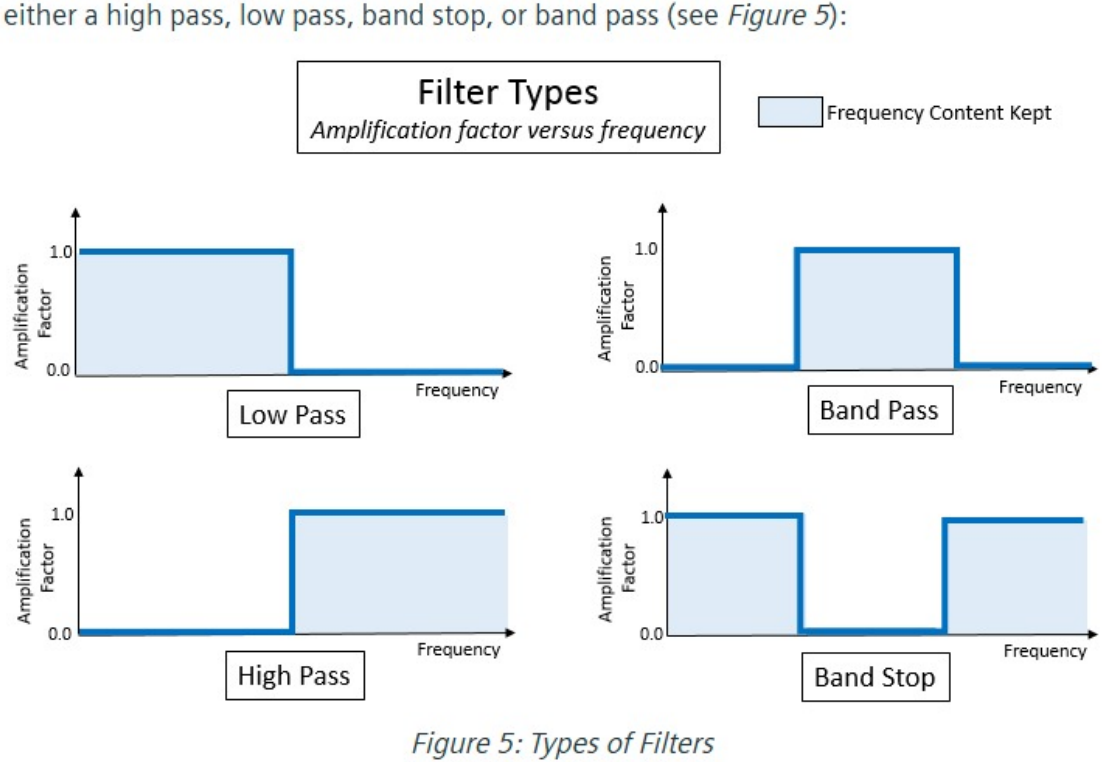


Figure 5: Types of Filters

Filter types:

- High Pass – High pass filter is used to remove low frequency offsets from a signal. For example, if interested in only the dynamic content of a strain gauge signal, any low frequency drift in a gauge could be removed with a high pass filter.
- Low Pass - A low pass filter attenuates, or gets rid of, frequencies above a specified frequency. For example, this could be used to remove high frequency hiss from an audio recording.
- Band Pass – This filter is used to allow only a band limited portion of the frequency content to be passed through the filter.
- Band Stop – A band stop filter is used to remove frequency content over a specified range.

These filter types can be implemented using either FIR or IIR filters. It is also possible to use a combination of these to produce an arbitrary shaped filter.

3. FIR versus IIR

Two classes of digital filters are Finite Impulse Response (FIR) and Infinite Impulse Response (IIR).

The term 'Impulse Response' refers to the appearance of the filter in the time domain. Filters typically have broad frequency responses, which correspond to short duration pulses in the time domain as shown in *Figure 6*.

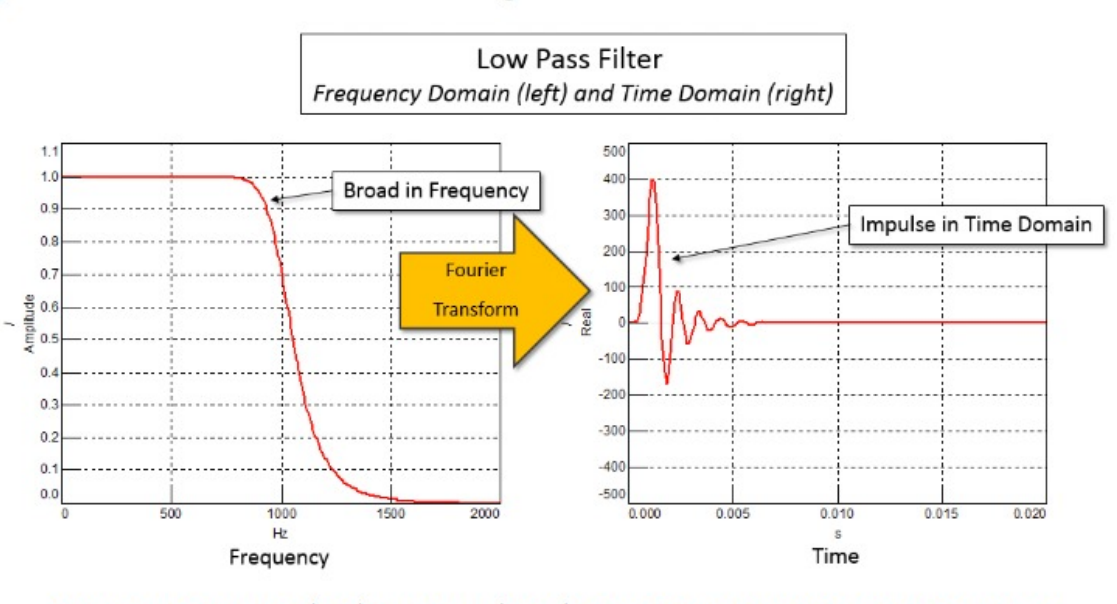


Figure 6: Frequency (left) and time (right) domain representations of filter shape.

The equations for both an IIR and FIR filter are shown in *Equation 1*. The input to the filter is time series x(n), and the output of the filter is a time series y(n). The first sample in the time history is at n=0.

FIR Filter Equation: $y(n) = \sum_{k=0}^{N} a(k)x(n-k)$

IIR Filter Equation: $y(n) = \sum_{k=0}^{N} a(k)x(n-k) + \underbrace{\sum_{j=0}^{P} b(j)y(n-j)}_{\text{Output used recursively}}$

Equation 1: Finite Impulse Response (FIR) filter equations versus Infinite Impulse Response (IIR) filter.

The mathematical difference between the IIR and FIR implementation is that the IIR filter uses some of the filter output as input. This makes the IIR filter a 'recursive' function.

Each equation has three series of numbers: an input time history, a filter, and an output time history (*Figure 7*).

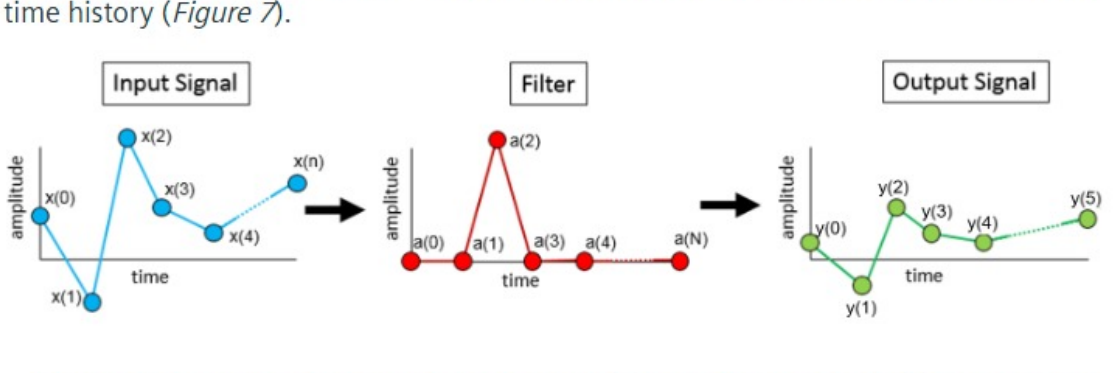
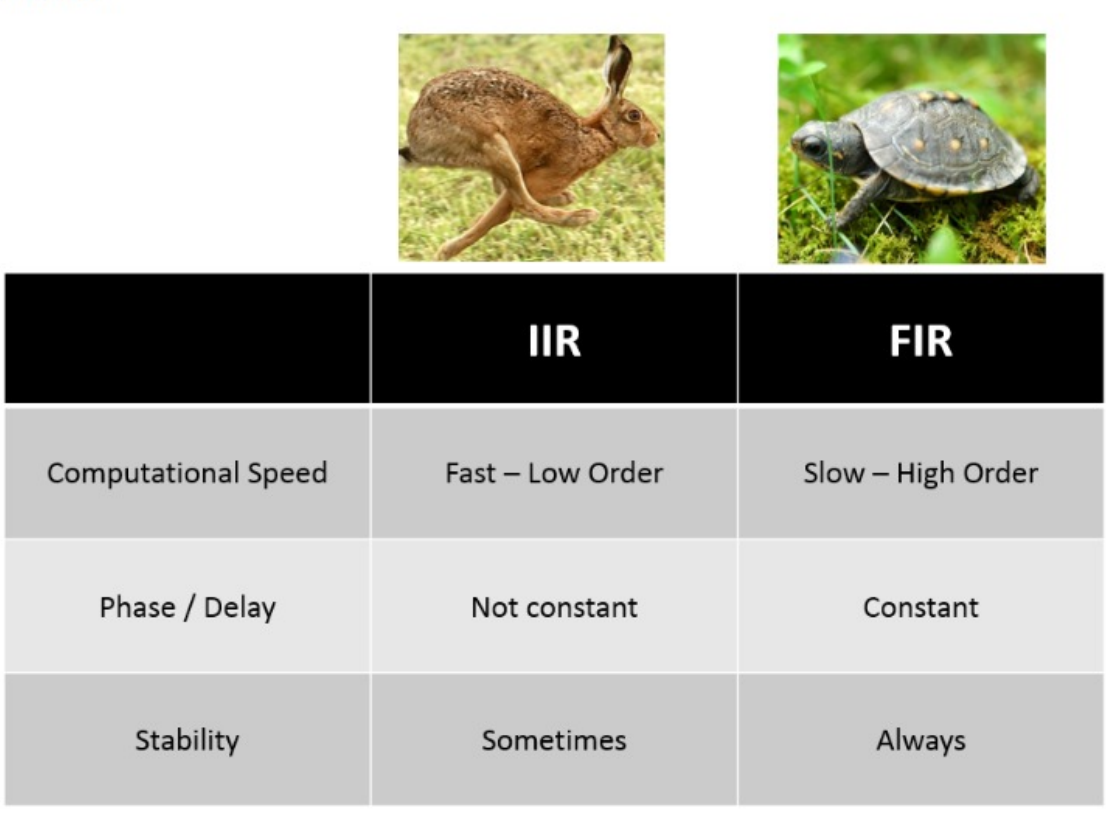


Figure 7: Time series for input time history, "x(n)", FIR filter - Series "a(k)", and output time history, "y(n)".

The series are:

- Series x(n) - The input time history is x(0), x(1), x(2), to x(*n*). Lower case *n* is the total number of data points in the input time history.
- Series a(k) - The filter is represented by the series labelled "a" for a FIR filter, and "a" and "b" for an IIR filter. Uppercase letters N and P are the number of terms in the filters respectively, which is also referred to as the filter "order" or "number of taps". For example, the FIR filter is performed from k=0 to k=N.
- Series y(n) - Output time history is y(0), y(1), y(2), …

In practice, FIR and IIR filters have important performance differences as summarized in *Figure 8*.



	IIR	FIR
Computational Speed	Fast – Low Order	Slow – High Order
Phase / Delay	Not constant	Constant
Stability	Sometimes	Always

Figure 8: Summary of differences between IIR and FIR filters.

An IIR filter has an advantage that for a similar filter roll off as a FIR, a lower order or number of terms can be used. This means that less computations are needed to achieve the same result, making the IIR faster computationally. However, an IIR has nonlinear phase and stability issues. It is a bit like the fable of the tortoise and the hare. The FIR filter is like the tortoise in the race – slow and steady, and always finishes. The hare is like the IIR filter – very fast, but sometimes crashes and does not complete the race.

These differences are discussed in more detail in the next sections.

4. Filter Order and Computational Speed

From the FIR filter equation in *Equation 1*, the higher the number N, the higher the "order" of the filter. For example, if there were 5 terms in the filter versus 10, the filter computations would take twice as long. However, the filter roll off would be sharper, as shown in *Figure 9*.

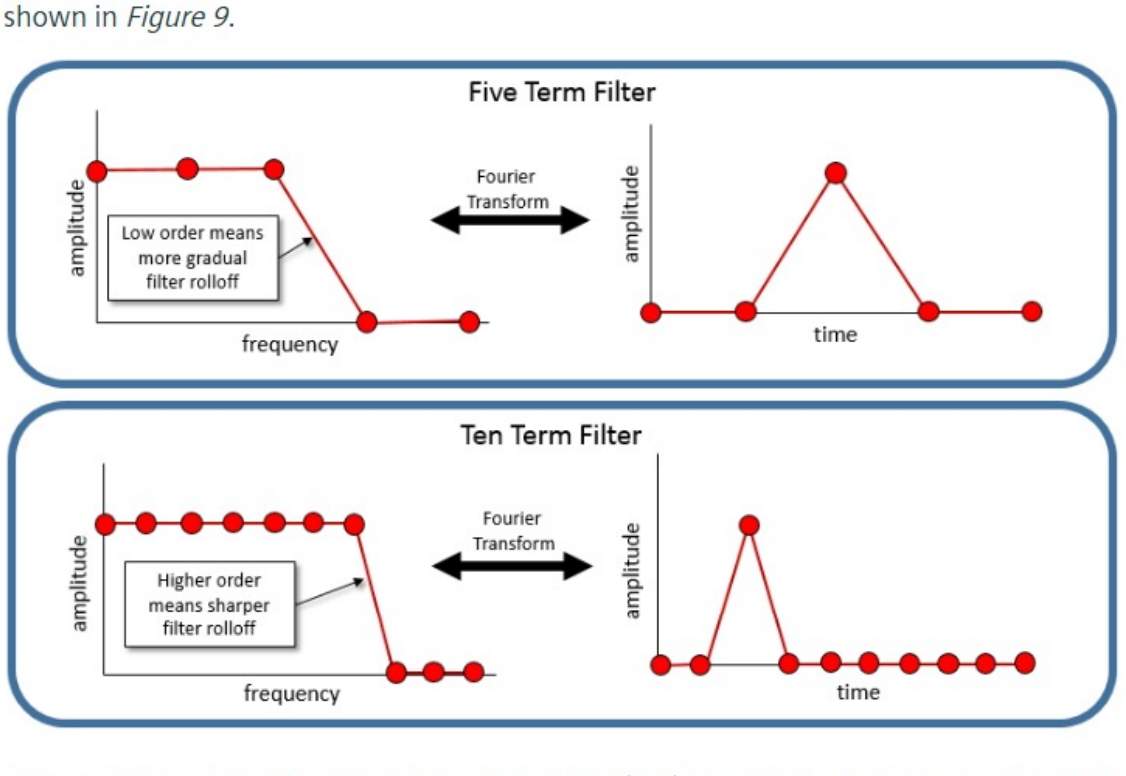


Figure 9: Top – The roll off sharpness of the filter (top) with fewer terms is less sharp than filter with more terms (bottom).

A filter with more terms (i.e., a higher order) has a sharper transition between the frequencies being passed and the frequencies being stopped.

An example comparison of the same filter type, calculated with different orders, is shown below in *Figure 10*.

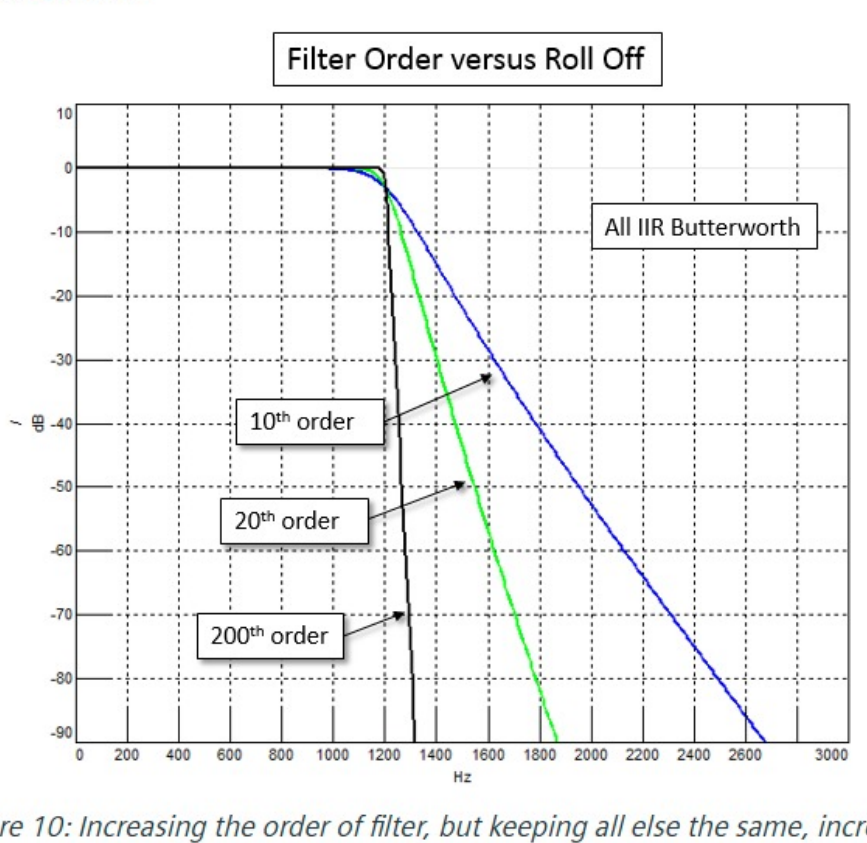


Figure 10: Increasing the order of filter, but keeping all else the same, increases the sharpness of the filter roll off.

Making a filter sharper is done by increasing the order. This requires more calculations, and has an impact on the time delay introduced by the filter.

> The higher the order, the sharper the filter.

The sharpness of FIR and IIR filters is very different for the same order as shown in *Figure 11*. Because of the recursive nature of an IIR filter (see *Equation 1*), where part of the filter output is used as input, it achieves a sharper roll off with the same order filter.

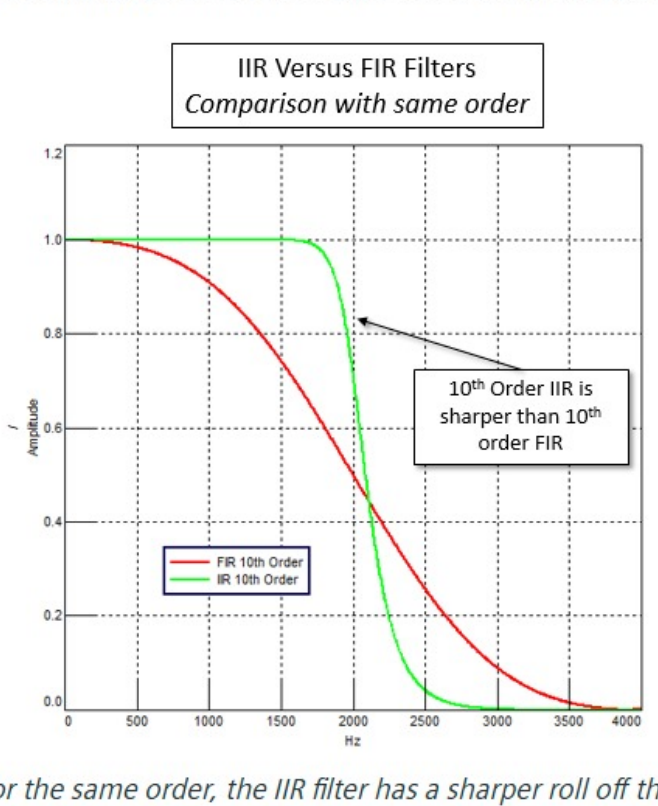


Figure 11: For the same order, the IIR filter has a sharper roll off than a FIR filter.

Conversely, fewer terms can be used in an IIR to achieve the same performance as a FIR filter. This is shown in *Figure 12*.

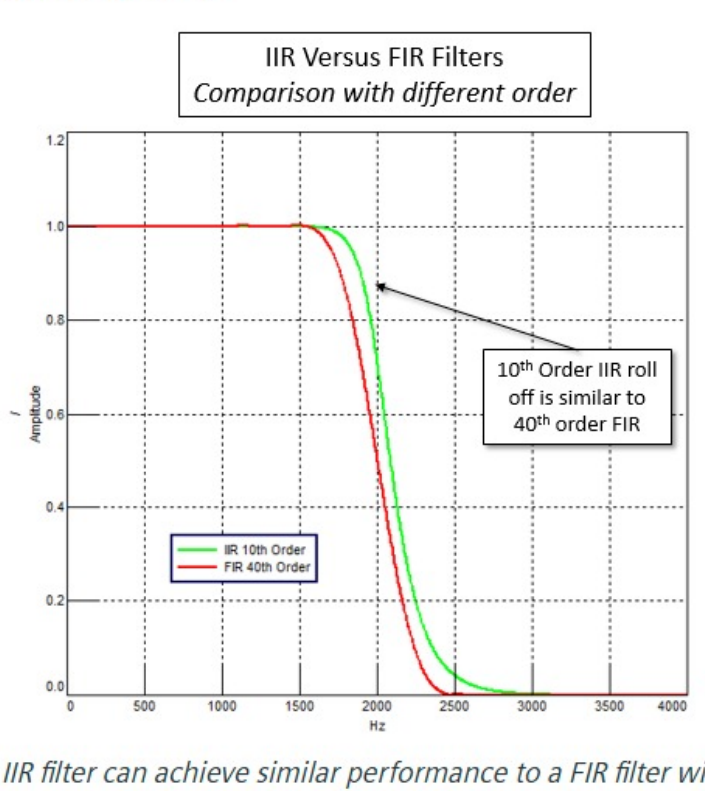


Figure 12: An IIR filter can achieve similar performance to a FIR filter with a lower order.

This makes IIR filters faster from a computational point of view than FIR filters. If a filter has to be implemented in real-time application (for example interactive filtering while listening), it is typically done with an IIR filter.

However, there are some potential drawbacks to using an IIR filter:

- Delays - An IIR filter has an unequal delay at different frequencies, while a FIR filter has a consistent delay at every frequency.
- Stability – Due to its construction, an IIR filter can sometimes be unstable and not be able to be computed or applied to the data. The FIR filter formulation is always stable.

> For the same order, an IIR Filter has a sharper roll off than a FIR filter.

5. Time Delay and Filters

After applying a filter to time data, and comparing to the original time history, a slight shift, or time delay, can be seen in the data. In *Figure* 13, simultaneously acquired sound (red curve, top) and vibration data (blue curve, bottom) are shown. Filtering the sound data (green curve, top) causes events between the sound and vibration to not be aligned.

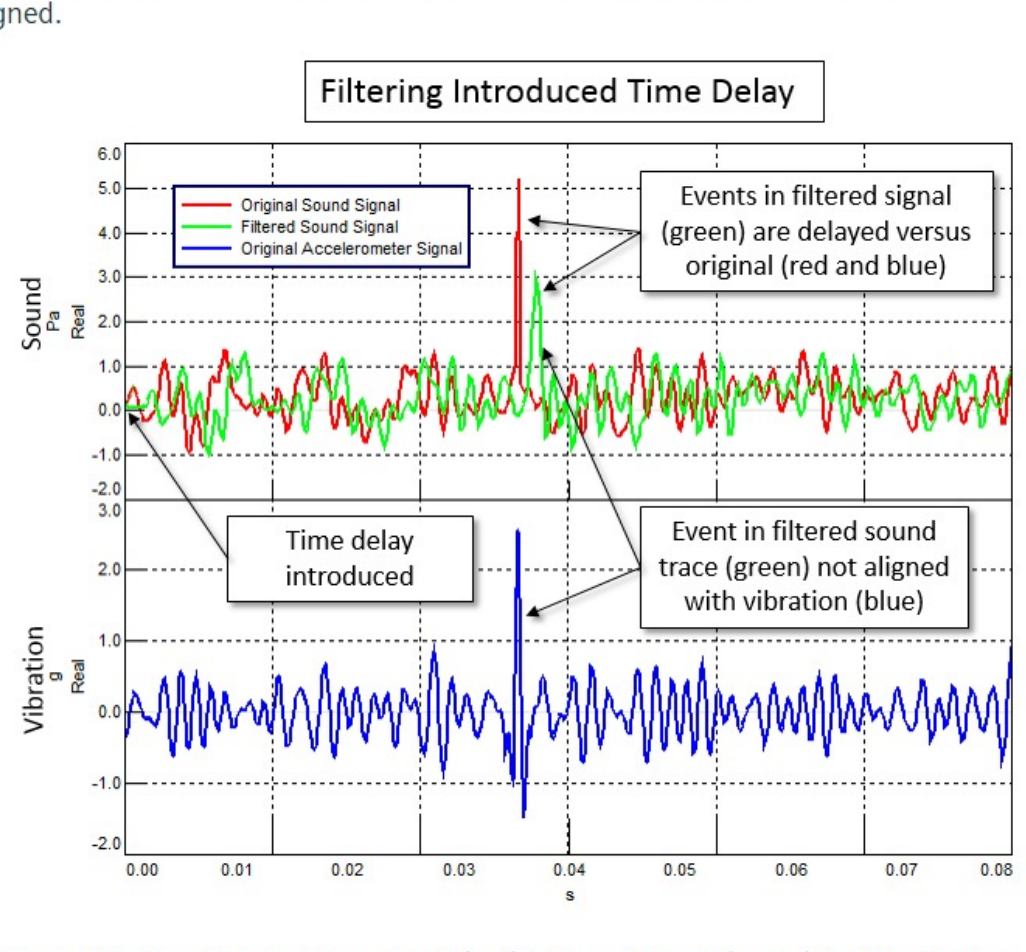


Figure 13: Top - Original time signal (red) is time delayed (green) by filter. Bottom – Vibration signal (blue) no longer aligned with filtered sound signal (green).

In some applications, this would not be a problem – the offset is known and can be ignored.

In other applications, this delay might be important. For example:

- Troubleshooting - With a multi-channel recording of sound and vibration, if a filter was applied only to the sound channel, a time delay would be introduced into the sound relative to the vibration channels (see bottom graph in *Figure 13*). When trying to figure out if a vibration event created a sound, this time misalignment would make it difficult to see if the vibration and sound events are related.
- Operational Deflection Shapes – If some vibration channels used in an operational deflection shape had a filter applied, and others did not, this would cause the phase relationship between those channels to be altered. As a result, the animation would not be correct.

What causes a filter to introduce a time delay in the output time history?

Take the equation for a FIR filter from *Figure 14*. Working through the equation, the cause for the time delay can be observed.

FIR Filter Equation: $y(n) = \sum_{k=0}^{N} a(k)x(n-k)$

For n = 0, k = 0 to N:
$y(0) = a(0)x(0) + a(1)x(-1) + a(2)x(-2) + a(3)x(-3) \ldots$

For n = 1, k = 0 to N:
$y(1) = a(0)x(1) + a(1)x(0) + a(2)x(-1) + a(3)x(-2) \ldots$

For n = 2, k = 0 to N:
$y(2) = a(0)x(2) + a(1)x(1) + a(2)x(0) + a(3)x(-1) \ldots$

Terms highlighted in orange do not exist, filter does not become fully realized until n > N

Figure 14: To be fully realized (i.e., actually work) the filter requires as many data points (n) as the terms (N) to be passed through the filter.

A number of time data samples from the input signal (x) must pass through the filter that is proportional to the number of terms (N), before the filter will work. The filtered output data does not start until the number (n) data points passed through the filter is greater than N.

Because some data must pass through the filter to even create an output, a delay in the output time history (y) is created when compared to the input time history (x) as shown in *Figure 13*. By making the filter sharper, this delay is increases.

> The sharper the filter, the longer the time delay.

The delay characteristics of a FIR and IIR filter are very different as shown in *Figure 15*.

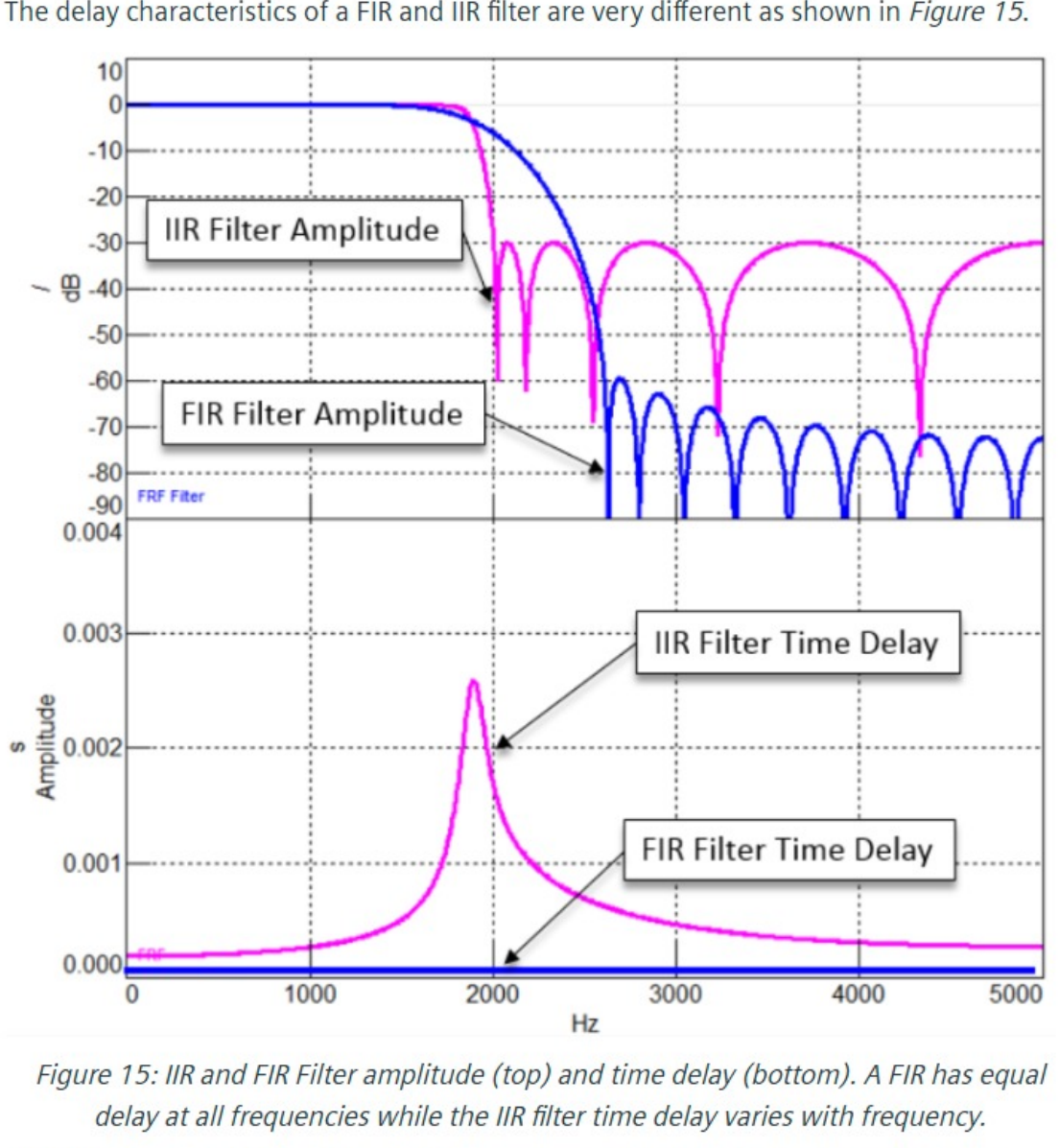


Figure 15: FIR and IIR filter amplitude (top) and time delay (bottom). A FIR has equal delay at all frequencies while the IIR time delay varies with frequency.

A FIR filter has equal time delay at all frequencies, while the IIR filter time delay varies with frequency. Usually the biggest time delay in the IIR filter is at the cut off frequency of the filter.

> FIR filters have equal time delay at all frequencies while a IIR filter has variable time delay at each frequency.

All filters create a delay of some sort – analog and digital. Depending on the filter characteristics, the delay can be shorter or longer. They can also be variable as a function of frequency.

5.1 Direct vs Zero Phase

The delay in the output time history can be eliminated by filtering the data 'forward and backward'. After the time history x(n) is filtered, and the new output time history y(n) is created, it can be fed backward into the filter. The data points in y(n) are reversed in order in time and fed into the filter again. This is referred to as "zero phase filtering". The result is shown in *Figure 16*.

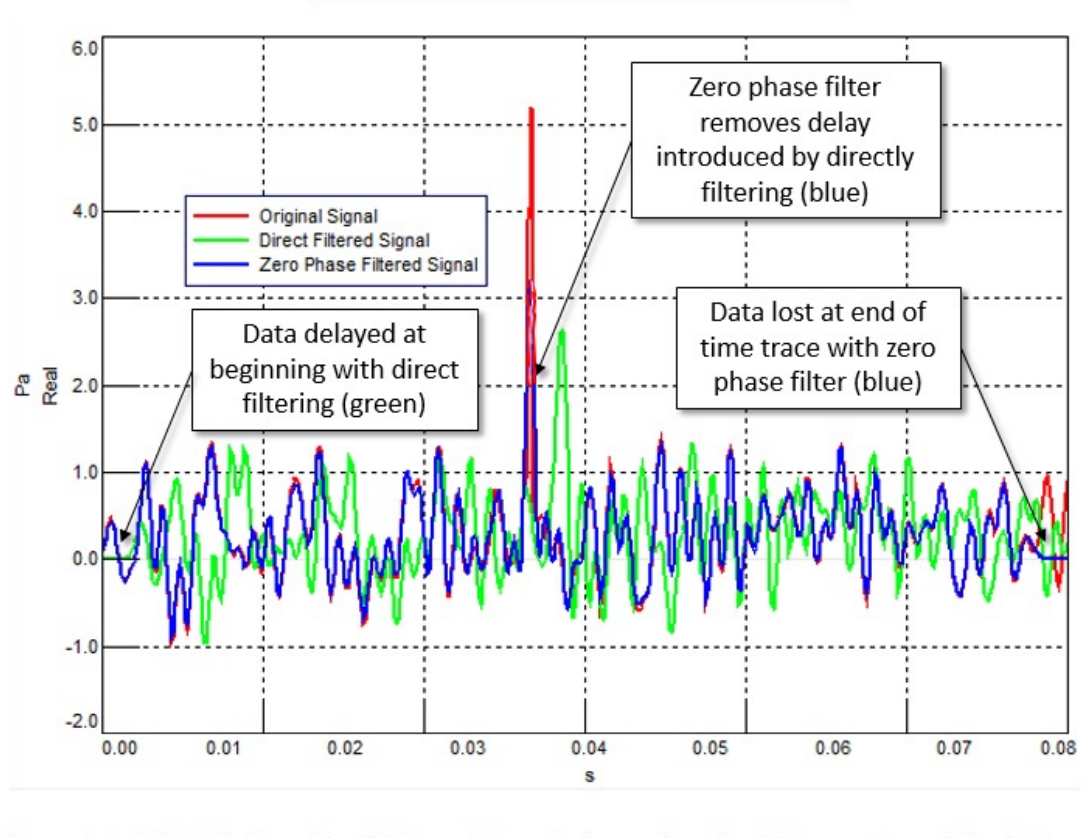


Figure 16: Original signal (red) filtered directly (green) and with zero phase (blue). Zero phase removes the delay, but data is lost at the end of zero phase filtered time history. Notice that the time delay is at the end of the zero-phase filtered trace (blue).

Not only is the output y(n) of the "zero phase" filter time aligned with x(n), the data is filtered twice, so the attenuation is doubled. There are trade-offs when using zero phase filters:

- Calculation takes twice as long to perform.
- Only easily done with digitized data, not easily done with analog signals.
- Data is eliminated at end of time trace.

> The delay and phase effects can be removed by filtering the time series forward and backward (zero phase filtering).

6. Filter Methods and Attributes

The coefficients, a(n), of a filter can be selected to control specific filter attributes. The coefficients, or methods, affect filter attributes shown in *Figure 17*.

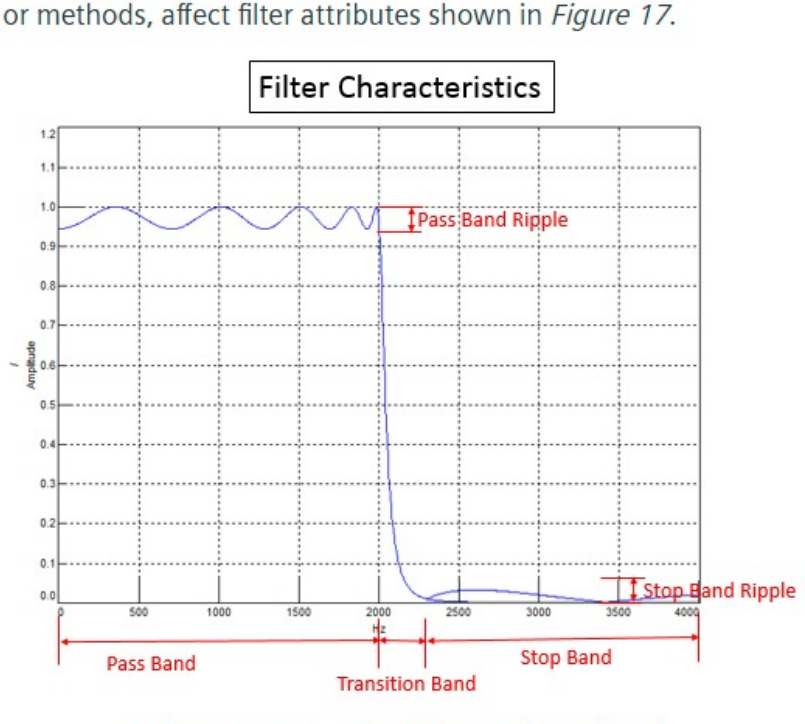


Figure 17: Pass, Transition, and Stop band.

There are four main filter attributes that methods affect:

- Pass Band – Data in the pass band is sent directly to the output time history. To ensure the data in the pass band is identical to the original time history data, there should be no ripple in the filter. Ripple is a slight variation in amplitude as function of frequency. Ideally, in this band, the filter should have an amplitude of exactly one.
- Transition Width – Depending on the application, it could be desirable to have the transition between the pass and stop bands be as narrow as possible in terms of frequency. Both the method and the filter order determine how rapidly the transition between pass and stop bands occurs.
- Stop Band – The stop band could also contain data if the filter has ripple. In some applications, the amplitude may be so small it does not matter. In others, the ripple might not be acceptable.
- Group Delay/Phase – Filters create a delay in the output time history, and this can even vary as a function of frequency. As previously explained, running a time history forward and backward through a filter removes this delay (Zero Phase filter). For some applications, where speed is critical and a zero phase filter operation is not an option, the phase can be important consideration.

6.1 FIR Filter Methods

Methods for Finite Impulse Response (FIR) filters are listed below, and shown in *Figure 18:*

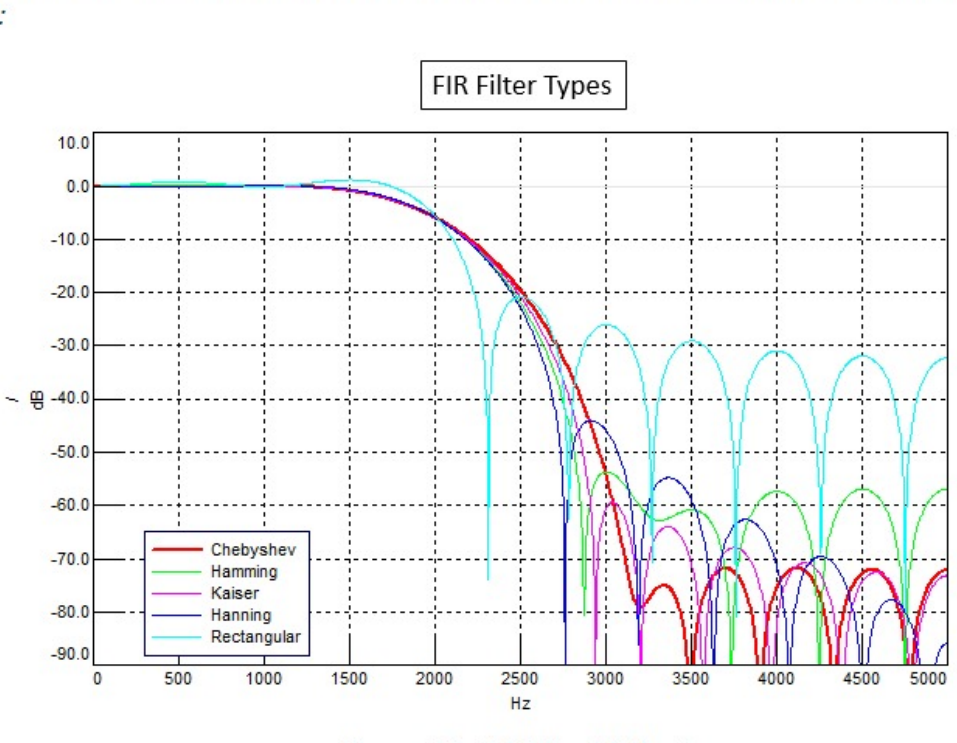


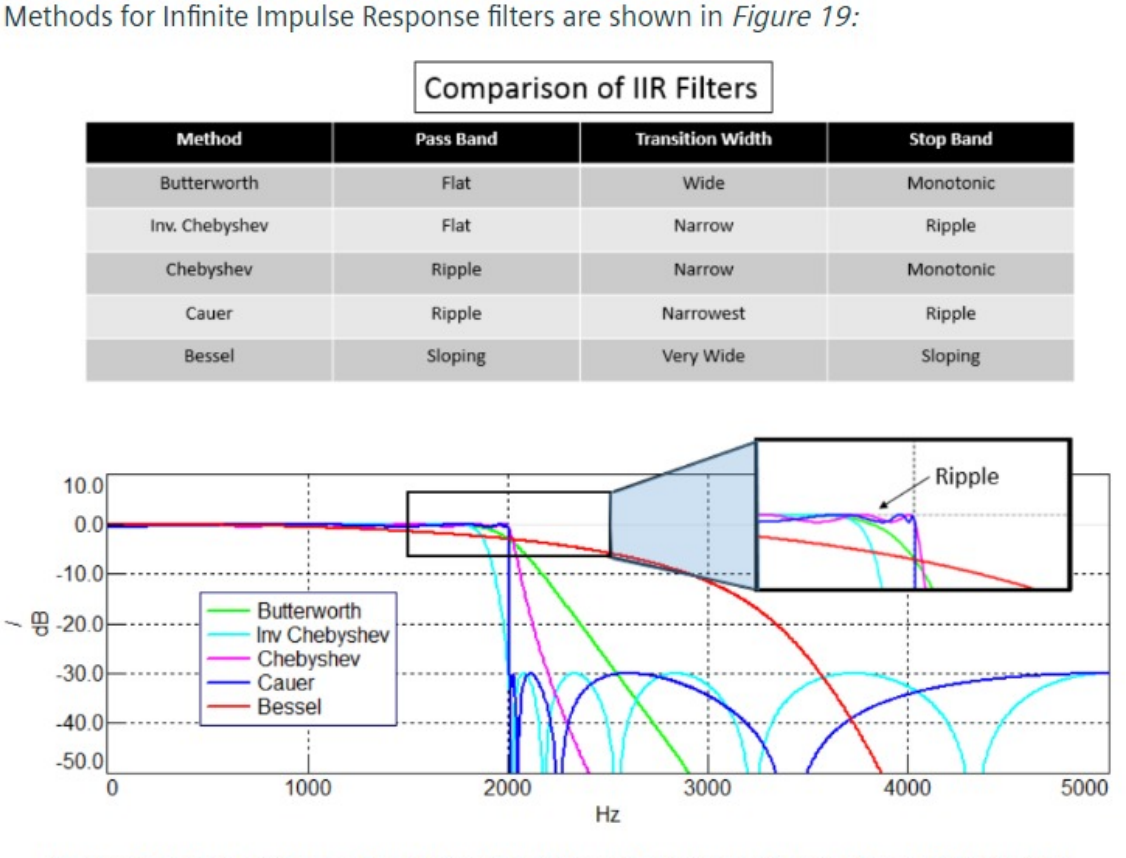
Figure 18: FIR Filter Methods.

The FIR methods use different spectral windows when transforming from the frequency to time domain. Some of the window methods include:

- Chebyshev – Has the lowest amount of ripple in stop band, but widest transition band.
- Hamming – Narrow transition zone, smaller ripple than Hanning. Developed by Richard Hamming, who was a member of the Manhattan Project.
- Kaiser – Developed by James Kaiser at Bell Laboratories, the Kaiser window has small amplitude ripple in stop zone, only the wide transition width Chebyshev has lower amplitude ripple.
- Hanning – Narrowest transition band, but large ripple in stop band.
- Rectangular – Largest amount of ripple/lobes, even affects pass band.

6.2 IIR Methods

Methods for Infinite Impulse Response filters are shown in *Figure 19:*



Comparison of IIR Filters

Method	Pass Band	Transition Width	Stop Band
Butterworth	Flat	Wide	Monotonic
Inv. Chebyshev	Flat	Narrow	Ripple
Chebyshev	Ripple	Narrow	Monotonic
Cauer	Ripple	Narrowest	Ripple
Bessel	Sloping	Very Wide	Sloping

Figure 19: Comparison of Low Pass IIR filters with same order and cutoff frequency.

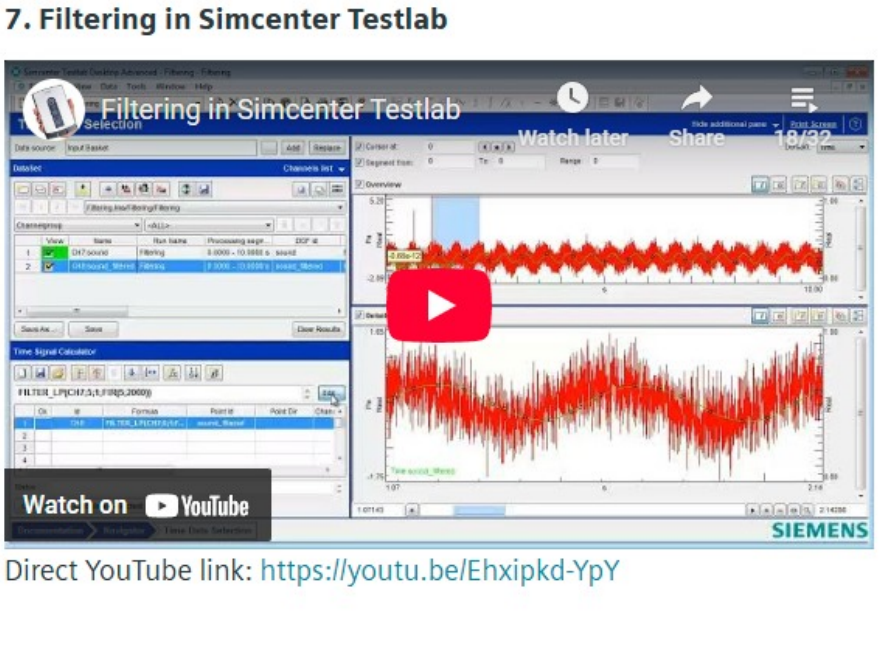
Attributes of the different IIR filter methods:

- Butterworth – Flat response in both the pass and stop band, but has a wide transition zone. First described by British physicist Stephen Butterworth in 1930.
- Inverse Chebyshev – Flat in the pass band, with a narrower transition width than the Butterworth filter, but has ripple in the stop band. If ripple in the stop band is not a problem, might be preferred for a given application over the Butterworth filter.
- Chebyshev – Can have ripple in pass band, but has steeper rolloff than Inverse Chebyshev.
- Cauer – Narrowest transition zone. Ripple in both stop and pass bands. Sometimes called an Elliptic filter.
- Bessel – Sloping amplitude in both the pass and stop band, with a very wide transition zone. The delay versus frequency in the filter is the flattest in this list. The Bessel filter was named for Freidrich Bessel (1784-1846), a German mathematician.

> The filter method impacts important filter characteristics like ripple and transition band width.

A filter method can be selected to best suit a particular application.

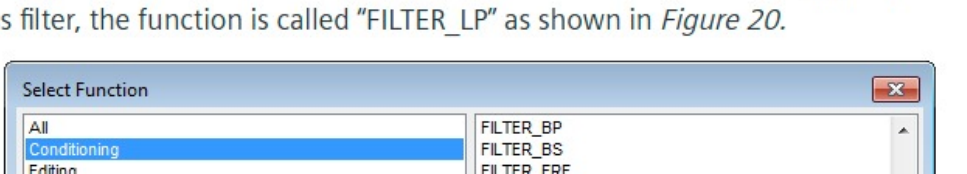
7. Filtering in Simcenter Testlab



Direct YouTube link: https://youtu.be/Ehsipkd-YpY

In Simcenter Testlab, filters can be applied to recorded signals using the Time Signal Calculator (Tools -> Add-ins -> Time Signal Calculator, 26 tokens) in the Time Data Selection worksheet.

The filtering capabilities of Time Signal Calculator are in the Conditioning group. To add a lowpass filter, the function is called "FILTER_LP" as shown in *Figure 20*.



FOLLOW
Files (0)

RELATED ARTICLES
- Simcenter Testlab Audio Replay and Filtering 1,440
- Read and Unread Filtering to Discussions 214
- Face Blend between Feature Intersection Edges in NX 11.0.1 5,319
- Discussion on numerical diffusion: Poly vs. Trimmed 1,590
- How-To: Use the Browser Filters 45
- Kurtosis 15,449
- Overlap: What, Why and How to use it 13,316